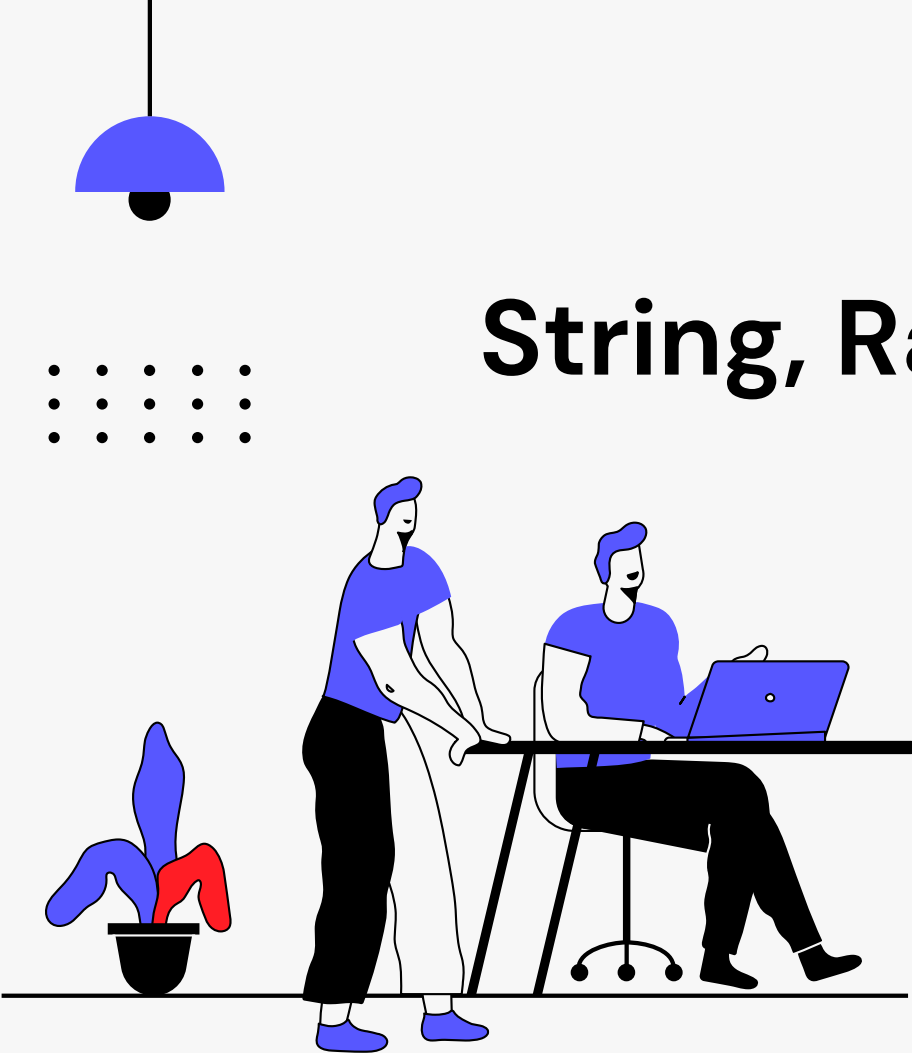


String, Random y Stream en **JAVA**

Introducción a la Programación
Orientada a Objetos





Clase String

- Clase provista por Java que brinda facilidades para almacenar y procesar **cadenas de caracteres**
- El **estado interno** de una instancia de tipo String es una secuencia de caracteres

```
String cad = "Buenos días..."
```

```
String cad = new String("Buenos días");
```

- En cualquiera de las dos alternativas, la variable *cad* puede recibir mensajes que se ligarán a métodos provistos por la clase String .

Clase String

<code>length()</code>	Retorna la cantidad de caracteres (longitud) de la cadena
<code>toLowerCase()</code>	Retorna la misma cadena en minúsculas
<code>toUpperCase()</code>	Retorna la misma cadena en mayúsculas
<code>trim()</code>	Elimina los espacios en blanco en ambos extremos de la cadena
<code>charAt(pos)</code>	Retorna el carácter que está ubicado en la posición <i>pos</i>

Clase String

<code>substring(ini)</code>	Retorna la subcadena a partir de la posición <i>ini</i>
<code>substring(ini, fin)</code>	Retorna la subcadena a partir de la posición <i>ini</i> , hasta la anterior a la posición <i>fin</i>
<code>indexOf(A)</code>	Retorna la posición de la primera aparición de la subcadena 'A' en la cadena (<i>retorna -1 si no aparece</i>)
<code>lastIndexOf(A)</code>	Retorna la posición de la última aparición de la subcadena 'A' en la cadena (<i>retorna -1 si no aparece</i>)
<code>compareTo(A)</code>	Retorna 0 si las cadenas son iguales, o un número positivo o negativo según el orden alfabético.

Clase String

- En Java el mínimo valor para un índice es **0** y corresponde al primer carácter de la cadena
- La comparación entre variables de tipo String **no** se realiza a través del **operador relacional** (**==**), sino con los métodos **equals(A)** o **compareTo(A)**

cad

B	U	E	N	O	S		D	I	A	S
0	1	2	3	4	5	6	7	8	9	10

Clase String

- **Mostrar por pantalla**

```
System.out.println(cad)
```

- **Concatenar cadenas**

```
System.out.println("El ganador es nombre"+cad);
```

- **Conversión implícita y Concatenación**

```
System.out.println("El puntaje es "+10);
```

- **Para convertir un String en un número entero :**

```
String cad ="12";
```

```
int numero = Integer.parseInt (cad);
```



Clase Random

- Un generador de números aleatorios se utiliza cuando se desea simular situaciones de azar
- La clase Random de Java es un **generador de números pseudo aleatorios**
- Los números no son realmente aleatorios porque se obtienen a través de un algoritmo que genera una secuencia distribuida uniformemente, a partir de una semilla inicial





Clase Random

- La clase brinda dos constructores para crear objetos Random

Sin parámetros

El constructor **usa la hora actual del sistema como semilla**, lo que disminuye la posibilidad de obtener secuencias de números repetidas

Con un parámetro

El parámetro establece el valor de la semilla. Se obtiene lo que parece una secuencia aleatoria, pero si se vuelve a inicializar el objeto con la misma semilla se vuelve a obtener la misma secuencia

Clase String

- Importar el paquete que incluye a la clase Random.

```
import java.util.Random;
```

- Crear un objeto de la clase Random

```
Random rnd = new Random();
```

```
Random rnd = new Random(100);
```

- Invocar uno de los métodos que generan un número aleatorio

```
int n1= rnd.nextInt(); //genera un número entero aleatorio
```

```
int n2= rnd.nextInt(3); //genera un número entero en el intervalo cerrado[0-2]
```

Método	Descripción	Rango
<code>rnd.nextInt()</code>	Número aleatorio entero de tipo <i>int</i>	2^{-32} y 2^{32}
<code>rnd.nextInt(n)</code>	Número aleatorio entero entre 0 y n-1	[0,n)
<code>rnd.nextLong()</code>	Número aleatorio entero de tipo <i>long</i>	2^{-64} y 2^{64}
<code>rnd.nextFloat()</code>	Número aleatorio real de tipo <i>float</i>	[0.0f , 1.0f)
<code>rnd.nextDouble()</code>	Número aleatorio real de tipo <i>double</i>	[0.0f , 1.0f)

<https://docs.oracle.com/javase/8/docs/api/java/util/Random.html>



Manejo de Archivos

- Para el manejo de archivos Java ofrece clases ya implementadas
- Para poder usar estas clases

```
import java.io.*;
```

- Para la lectura desde un archivo de texto se usan dos clases

FileReader

BufferedReader



Manejo de Archivos

- Declarar un objeto capaz de recibir mensajes que provoquen la lectura del archivo

```
BufferedReader br = null;
```

- Abrir el archivo para lectura

```
FileReader fr = new FileReader(ruta); //donde ruta es la dirección del archivo en el disco
```

- Crear el objeto BufferedReader y ligarlo a la variable *br*

```
br = new BufferedReader(fr);
```

- También se puede hacer todo junto

```
BufferedReader br = new BufferedReader (new FileReader(ruta));
```

Manejo de Archivos

- Para leer del archivo:

`String str = br.readLine() ;` //Lee una línea del archivo y la almacena en la cadena de caracteres ligada a la variable str

- Para cerrar un archivo:

`br.close() ;` //Cierra el archivo ligado a la variable br.

Importante

La lectura y escritura de archivos puede provocar errores en tiempo de ejecución. Es por eso que, durante su manipulación, Java nos obliga a manejar posibles excepciones mediante el uso de bloques **try-catch**

Manejo de Archivos

...

```
try{  
    FileReader fr = new FileReader(ruta);  
    BufferedReader br = new BufferedReader(fr);  
    String str = br.readLine();  
    br.close();  
}  
catch (IOException err) {  
    System.out.println("Error al procesar el archivo");  
}
```

Manejo de Archivos

Ejemplo: Implementar un método que lea una secuencia de números enteros de un archivo y compute la cantidad de elementos leídos.

```
import java.io.*;
class TesterSecuencia{

    public static void main (String [] args){
        String ruta = "secuencia.txt";
        int cont = procesarArch(ruta) ;
        System.out.println("La cantidad de elementos de la
        secuencia es:" +cont);
    }
}
```

Manejo de Archivos

```
public static int procesarArch(String ruta){
    int cont =0;
    try{
        FileReader fr = new FileReader(ruta);
        BufferedReader br = new BufferedReader(fr);
        String str;
        while((str = br.readLine()) != null)
            cont++;
        br.close();
    }
    catch(IOException err){
        System.out.println("Error al leer el archivo"); }
    return cont;
}}
```


¿Preguntas?

